

REST/JSON APIs

`FastHttpUser` provides a ready-made `rest` method, but you can also do it yourself:

```
from json import JSONDecodeError
...
with self.client.post("/", json={"foo": 42, "bar": None}, catch_response=True) as response:
    try:
        if response.json()["greeting"] != "hello":
            response.failure("Did not get expected value in greeting")
    except JSONDecodeError:
        response.failure("Response could not be decoded as JSON")
    except KeyError:
        response.failure("Response did not contain expected key 'greeting'")
```

=====

Grouping requests

It's very common for websites to have pages whose URLs contain some kind of dynamic parameter(s). Often it makes sense to group these URLs together in User's statistics. This can be done by passing a *name* argument to the `HttpSession's` different request methods.

Example:

```
# Statistics for these requests will be grouped under: /blog?id=[id]
for i in range(10):
    self.client.get("/blog?id=%i" % i, name="/blog?id=[id]")
```

There may be situations where passing in a parameter into request function is not possible, such as when interacting with libraries/SDK's that wrap a Requests session. An alternative way of grouping requests is provided by setting the `client.request_name` attribute.

```
# Statistics for these requests will be grouped under: /blog?id=[id]
self.client.request_name="/blog?id=[id]"
for i in range(10):
    self.client.get("/blog?id=%i" % i)
self.client.request_name=None
```

=====

Grouping requests

It's very common for websites to have pages whose URLs contain some kind of dynamic parameter(s). Often it makes sense to group these URLs together in User's statistics. This can be done by passing a *name* argument to the `HttpSession's` different request methods.

Example:

```
# Statistics for these requests will be grouped under: /blog/?id=[id]
for i in range(10):
    self.client.get("/blog?id=%i" % i, name="/blog?id=[id]")
```

There may be situations where passing in a parameter into request function is not possible, such as when interacting with libraries/SDK's that wrap a Requests session. An alternative way of grouping requests is provided by setting the `client.request_name` attribute.

```
# Statistics for these requests will be grouped under: /blog/?id=[id]
self.client.request_name="/blog?id=[id]"
for i in range(10):
    self.client.get("/blog?id=%i" % i)
self.client.request_name=None
```

=====

Very useful feature for grouping requests that are related to the complete checkout workflow in the e-commerce applications:

If you want to chain multiple groupings with minimal boilerplate, you can use the `client.rename_request()` context manager.

```
@task
def multiple_groupings_example(self):
    # Statistics for these requests will be grouped under: /blog/?id=[id]
    with self.client.rename_request("/blog?id=[id]"):
        for i in range(10):
            self.client.get("/blog?id=%i" % i)

    # Statistics for these requests will be grouped under: /article/?id=[id]
    with self.client.rename_request("/article?id=[id]"):
        for i in range(10):
            self.client.get("/article?id=%i" % i)
```

=====

Connection reuse

By default, connections are reused by an `HttpUser`, even across tasks runs. To avoid connection reuse you can do:

```
self.client.get("/", headers={"Connection": "close"})
self.client.get("/new_connection_here")
```

Or you can close the entire requests.Session object (this also deletes cookies, closes the SSL session etc). This has some CPU overhead (and the response time of the next request will be higher due to SSL renegotiation etc), so don't use this unless you really need it.

```
self.client.get("/")
self.client.close()
self.client.get("/new_connection_here")
```

=====

```
from locust import HttpUser, task
from urllib3 import PoolManager
```

```
class TestConnectionPool(HttpUser):
    pool_manager = PoolManager(maxsize=100, block=True)
    @task
    def get_hello_world(self):
        self.client.get('/')
        =====
```

Connection pooling

As every `HttpUser` creates new `HttpSession`, every user instance has its own connection pool. This is similar to how real users (browsers) would interact with a web server.

If you instead want to share connections, you can use a single pool manager. To do this, set `pool_manager` class attribute to an instance of `urllib3.PoolManager`.

```
from locust import HttpUser
from urllib3 import PoolManager

class MyUser(HttpUser):
    # All instances of this class will be limited to 10 concurrent connections at most.
    pool_manager = PoolManager(maxsize=10, block=True)
```

=====

- Project root
 - `common/`
 - `__init__.py`
 - `auth.py`
 - `config.py`
 - `locustfile.py`
 - `requirements.txt` (External Python dependencies is often kept in a requirements.txt)

A project with multiple locustfiles could also keep them in a separate subdirectory:

- Project root
 - `common/`
 - `__init__.py`
 - `auth.py`
 - `config.py`
 - `my_locustfiles/`
 - `api.py`
 - `website.py`
 - `requirements.txt`

=====

Environment Variables

Options can also be set through environment variables. They are typically the same as the command line argument but capitalized and prefixed with `LOCUST_`:

On Linux/macOS:

```
$ LOCUST_LOCUSTFILE=custom_locustfile.py locust
```

On Windows:

```
> set LOCUST_LOCUSTFILE=custom_locustfile.py
> locust
```

=====

Configuration File 🔗

Options can also be set in a configuration file in the [config](#) or [TOML file format](#).

Locust will look for `~/.locust.conf`, `./locust.conf` and `./pyproject.toml` by default. You can specify an additional file using the `--config` flag.

```
$ locust --config custom_config.conf
```

Here's an example:

locust.conf

```
locustfile = locust_files/my_locust_file.py
headless = true
master = true
expect-workers = 5
host = https://target-system
users = 100
spawn-rate = 10
run-time = 10m
tags = [Critical, Normal]
```

Have a look later in this page for [All available configuration options](#)

=====

pyproject.toml

When using a TOML file, configuration options should be defined within the `[tool.locust]` section.

```
[tool.locust]
locustfile = "locust_files/my_locust_file.py"
headless = true
master = true
expect-workers = 5
host = "https://target-system"
users = 100
spawn-rate = 10
run-time = "10m"
tags = ["Critical", "Normal"]
```

Note

Configuration values are read (and overridden) in the following order:

```
./pyproject.toml -> ./locust.conf -> (file specified using --config) -> env vars -> cmd args
```

=====

For all configuration: <https://docs.locust.io/en/stable/configuration.html#all-available-configuration-options>.

=====

Using multiple Locustfiles at once

`-f/--locustfile` accepts multiple, comma-separated locustfiles.

Example:

With the following file structure:

```
├─ locustfiles/
│   ├── locustfile1.py
│   ├── locustfile2.py
│   └─ more_files/
│       ├── locustfile3.py
│       └─ _ignoreme.py
```



```
$ locust -f locustfiles/locustfile1.py,locustfiles/locustfile2.py,locustfiles/more_files/locustfile3.py
```



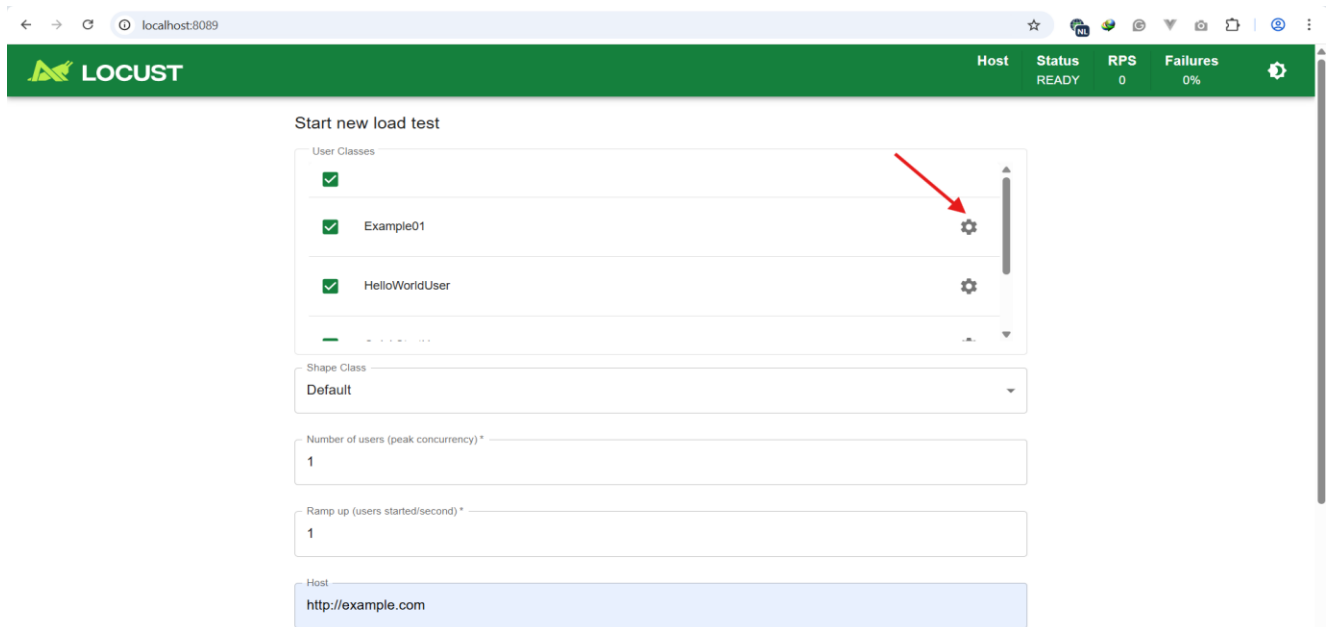
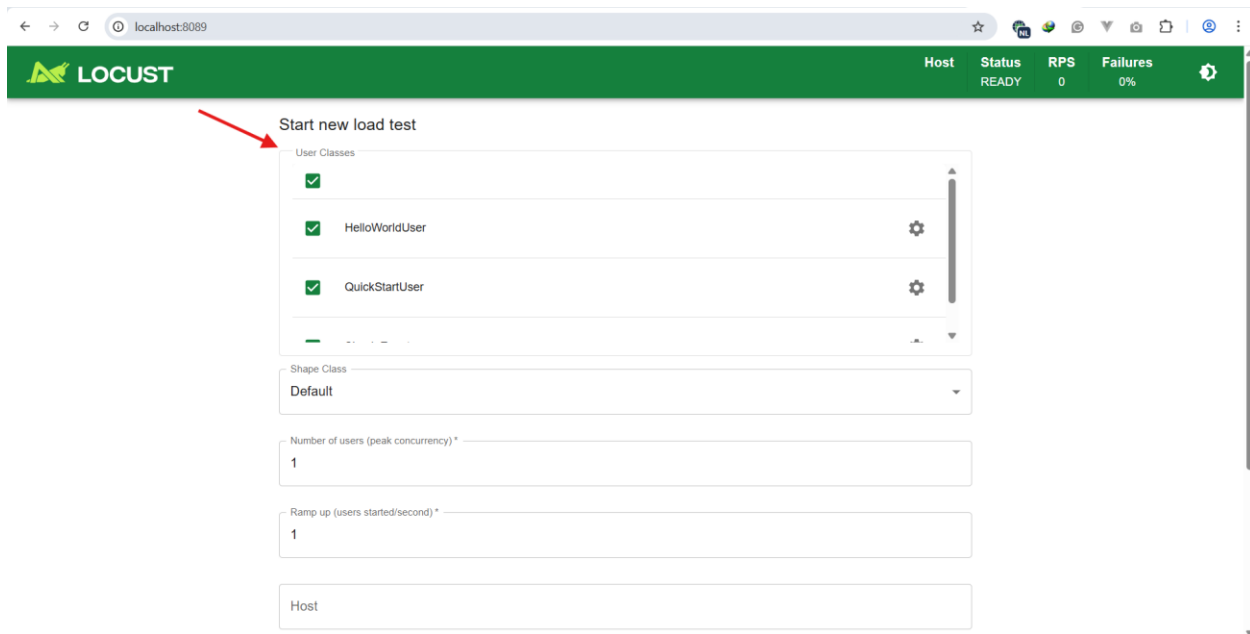
Locust will use `locustfile1.py`, `locustfile2.py` & `more_files/locustfile3.py`

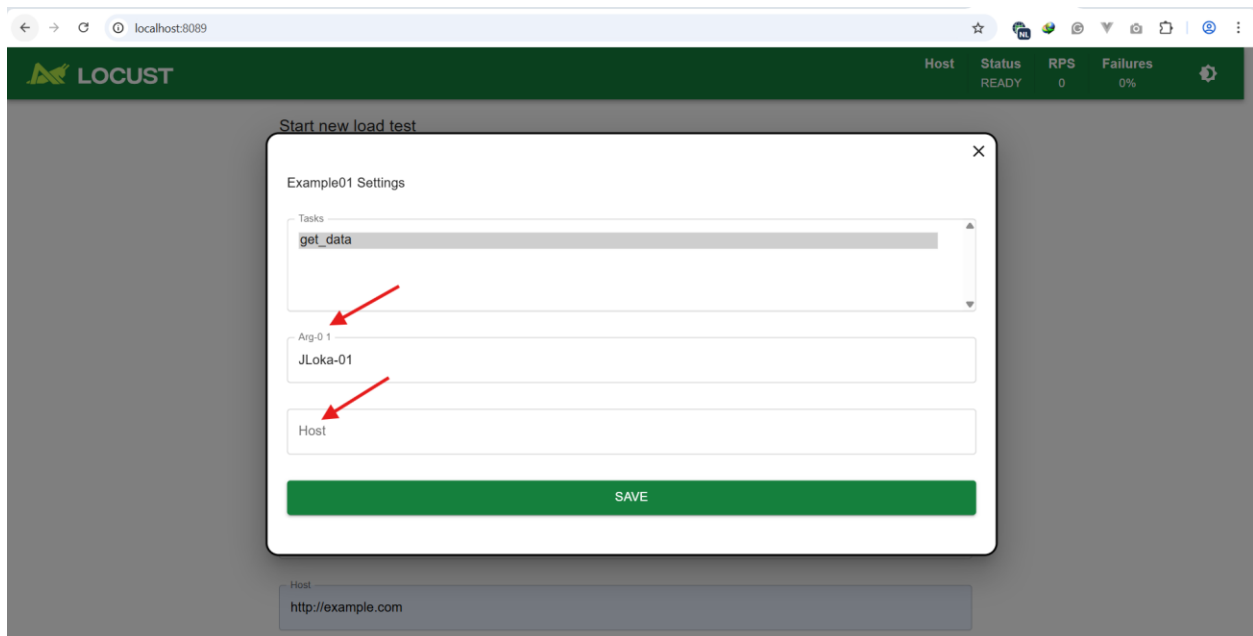
Additionally, `-f/--locustfile` accepts directories as an option. Locust will recursively search specified directories for `*.py` files, ignoring files that start with “_”. 

=====

```
(.venv) C:\Tests\j-l-qa-testing\Python-Locust>locust -f simple_test --class-picker
[2026-04-29 17:22:00,371] Jafar/INFO/locust.main: Starting Locust 2.43.4
[2026-04-29 17:22:00,372] Jafar/INFO/locust.main: Starting web interface at http://localhost:8089, press enter to open your default browser.
I'm on a worker or standalone node
```

=====





=====

```
from locust import task, HttpUser
```

```
class Example01(HttpUser):
```

```
    @task
```

```
    def get_data(self):
        self.client.get("/")
```

```
    @classmethod
```

```
    def json(cls):
        return {
            "host": cls.host,
            "arg-01": "JLoka-01"
        }
```

=====

It is even possible to add custom attributes that you wish to be configurable for each User. Simply add a `json` classmethod to your user:

```
class Example(HttpUser):
    @task
    def example_task(self):
        self.client.get(f"/example/{self.some_custom_arg}")

    @classmethod
    def json(self):
        return {
            "host": self.host,
            "some_custom_arg": "example"
        }
```

=====

Configure Users from command line

You can update User class attributes from the command line too, using the `--config-users` argument:

```
locust --config-users '{"user_class_name": "Example", "fixed_count": 1, "some_custom_attribute": false}'
```

To configure multiple users you pass multiple arguments to `--config-users`, or use a JSON Array. You can also pass a path to a JSON file.

```
$ locust --config-users '{"user_class_name": "Example", "fixed_count": 1}' '{"user_class_name": "Example", "fixed_count": 2}'
$ locust --config-users '["user_class_name": "Example", "fixed_count": 1], {"user_class_name": "Example", "fixed_count": 2}]'
$ locust --config-users my_user_config.json
```

=====

```
locust --config-users '{"user_class_name": "Example", "fixed_count": 1, "some_custom_attribute": false}'
locust --config-users '{"user_class_name": "Example", "fixed_count": 1}' '{"user_class_name": "ExampleTwo", "fixed_count": 2}'
locust --config-users '["user_class_name": "Example", "fixed_count": 1], {"user_class_name": "ExampleTwo", "fixed_count": 2}]'
locust --config-users my_user_config.json
```

=====

When using this way to configure your users, you can set any attribute.

Note

`--config-users` is a somewhat experimental feature and the json format may change even between minor Locust revisions.